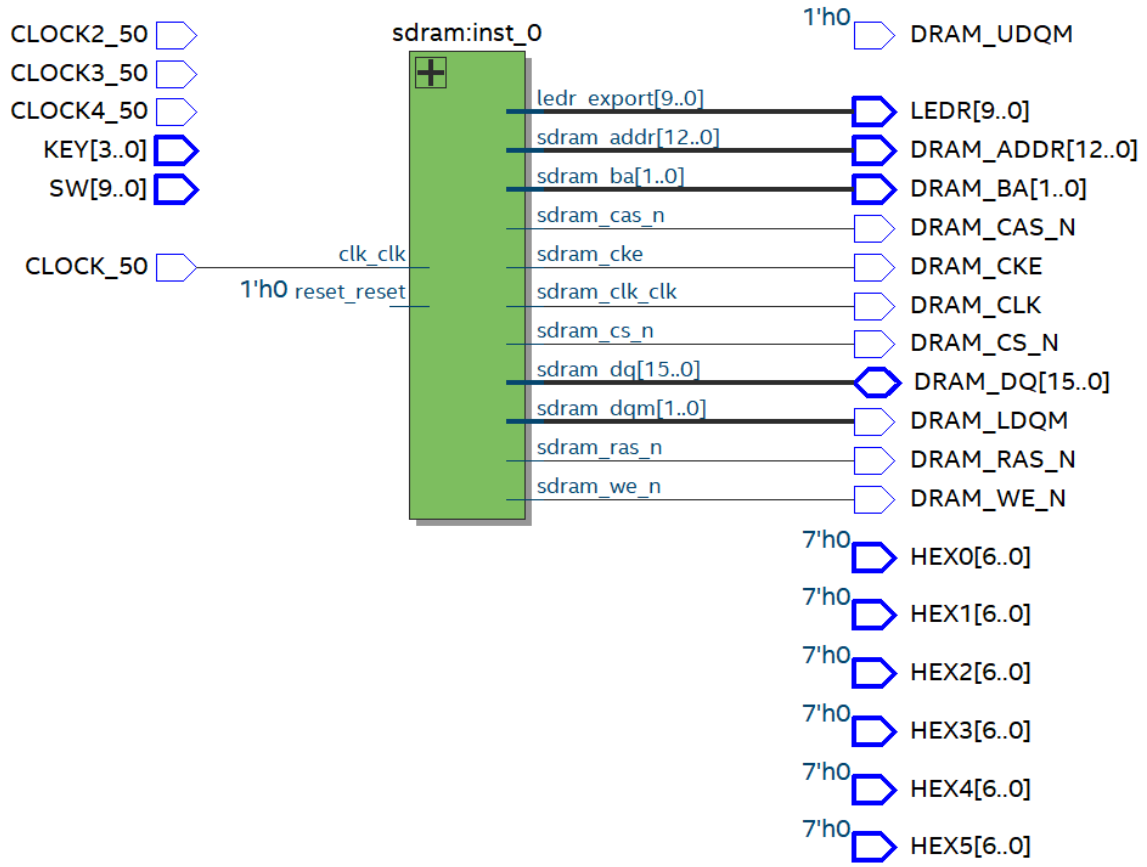
LAB 3

Giữ lại nội dung lab3 từ report lab 0_1_2_3
Chỉ đổi nội dung code C, RTL Viewer, và thêm hình ảnh minh chứng

RTL Viewer



Code C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <inttypes.h>

#define SIZE 10

int main () {
int A[SIZE];
```

```
int B[SIZE];
srand(time(NULL));

for (int i = 0; i < SIZE; i++) {
	// rand() % (max - min + 1) + min  -> Range from [-1000:1000]
	A[i] = rand() % (2001) - 1000;
}

// Copy A to B
for (int i = 0; i < SIZE; i++) {
	B[i] = A[i];
}

// bubble sort
for (int i = 0; i < SIZE - 1; i++) {
	for (int j = 0; j < SIZE - i - 1; j++) {
		if (B[j] > B[j + 1]) {
			int temp = B[j];
			B[j] = B[j + 1];
			B[j + 1] = temp;
		}
	}
}

// In  A
printf("Array A (original):\n");
for (int i = 0; i < SIZE; i++) {
	printf("A[%d] = %" PRId16 "\n", i, A[i]);
}

// In B
printf("\nArray B (sorted):\n");
for (int i = 0; i < SIZE; i++) {
	printf("B[%d] = %" PRId16 "\n", i, B[i]);
}

return 0;
}
```

Kết quả

Nios II Window Help

hello_world.c | system.h

```c
#include <inttypes.h>

#define SIZE 10

int main () {
int A[SIZE];
int B[SIZE];
srand(time(NULL));

for (int i = 0; i < SIZE; i++) {
    // rand() % (max - min + 1) + min    -> Range from [-100
    A[i] = rand() % (2001) - 1000;
}

// Copy A to B
for (int i = 0; i < SIZE; i++) {
    B[i] = A[i];
}

// bubble sort
for (int i = 0; i < SIZE - 1; i++) {
```

Problems | Tasks | Console | Nios II Console | Properties

lab4 Nios II Hardware configuration - cable: DE-SoC on localhost [USB-1] device ID: 2 instance ID: 0 name: jtag_uart_0.jtag

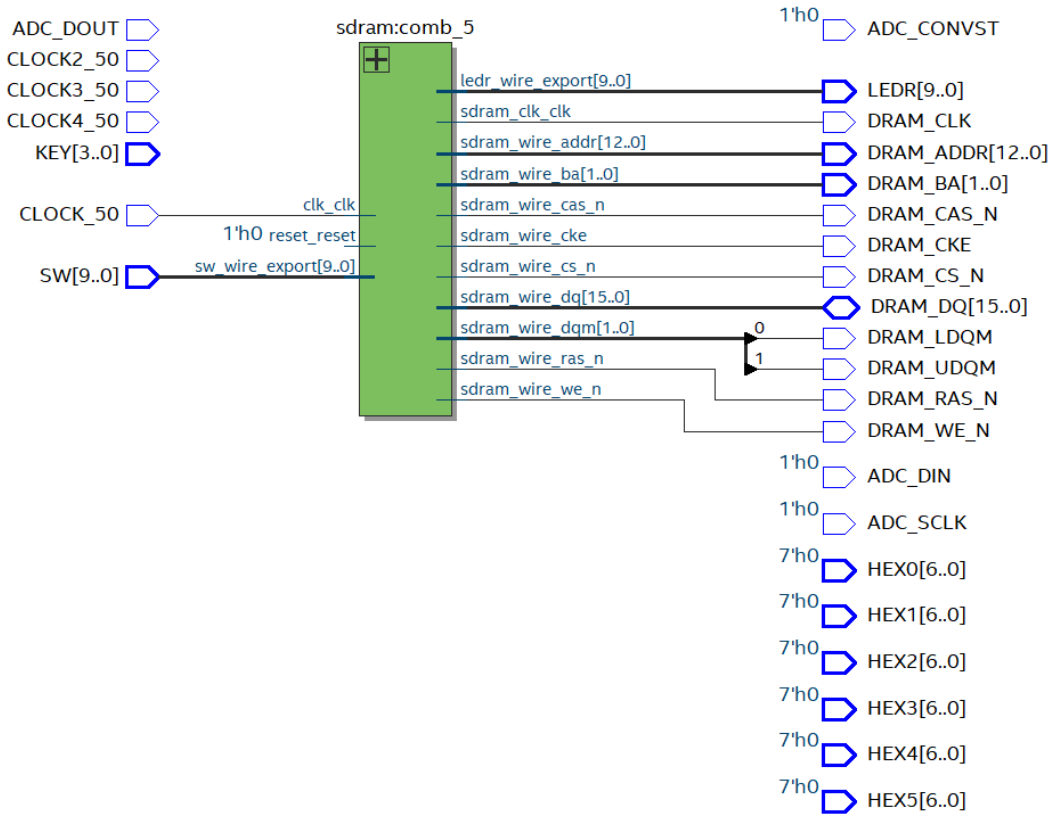
```
Array A (original):
A[0] = -1000
A[1] = 421
A[2] = 326
A[3] = 472
A[4] = -869
A[5] = 569
A[6] = -91
A[7] = 189
A[8] = 149
A[9] = -770

Array B (sorted):
B[0] = -1000
B[1] = -869
B[2] = -770
B[3] = -91
B[4] = 149
B[5] = 189
B[6] = 326
B[7] = 421
B[8] = 472
B[9] = 569
```

Search

ASUS Vivobook

LAB 4 - 5 (Chung)

RTL View



Resource

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	1942
2		
3	Combinational ALUT usage for logic	2922
1	-- 7 input functions	80
2	-- 6 input functions	577
3	-- 5 input functions	609
4	-- 4 input functions	494
5	-- <=3 input functions	1162
4		
5	Dedicated logic registers	2869
6		
7	I/O pins	113
8	Total MLAB memory bits	0
9	Total block memory bits	1703808
10		
11	Total DSP Blocks	3
12		
13	Total PLLs	2
1	-- PLLs	2
14		
15	Maximum fan-out node	sdram:comb_5|sdram_sys_sdram_pll_0...ra_pll:altera_pll_i|outclk_wire[0]
16	Maximum fan-out	3114
17	Total fan-out	31512
18	Average fan-out	4.82

Fmax (Condition Slow 1100mV 85C)

	Fmax	Restricted Fmax	Clock Name	Note
1	54.39 MHz	54.39 MHz	altera_reserved_tck	
2	86.57 MHz	86.57 MHz	comb_5|sys_sdram_pll_0|sys_pll|a...].gpll~PLL_OUTPUT_COUNTER|divclk	

Exercise 1:

Code C

```
#include <stdio.h>
#include <alt_types.h>
#include <sys/alt_alarm.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"

// Define BASE address
#define LED_BASE 0x04081050
#define SW_BASE 0x04081040


static alt_alarm alarm;
```

```
static int control = 0x01;

alt_u32 led_alarm_callback(void* context) {
   IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, control);
        control = control << 1 | 0x01;
        if (control == 2047) { // 0x11111111111
                control = 0x00;
        }

        // Return tick for 500ms
   return alt_ticks_per_second() / 2;
}

int main() {
   if (alt_alarm_start(&alarm, alt_ticks_per_second() / 2, led_alarm_callback,
NULL) < 0) {
       printf("No system clock available\n");
       return -1;
   }

   while (1) {
   }

   return 0;
}
```

Ex1.mp4

Exercise 2:

Code C

```
#include <stdio.h>
#include <alt_types.h>
#include <sys/alt_alarm.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"

// Define BASE address
#define LED_BASE 0x04081050
#define SW_BASE 0x04081040
```

```
static alt_alarm alarm1, alarm3, alarm5, alarm7, alarm9;
static int led_state[10] = {0};
static int is_SW1_SET = 0;

void set_led(int index, int state) {
	if (is_SW1_SET == 1) {
		int mask = (1 << index);
		int current = IORD_ALTERA_AVALON_PIO_DATA(LED_BASE);
		if (state)
			current |= mask;
		else
			current &= ~mask;
		IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, current);
		led_state[index] = state;
	}
	else {
		IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0x00); // OFF
all LED
		for (int i = 0; i < 10; i++) led_state[i] = 0;
	}
}

alt_u32 cb_led1(void* context) {
   set_led(0, !led_state[0]); // LED1
   return alt_ticks_per_second() * 2; // 0.25Hz = 4s
}
alt_u32 cb_led3(void* context) {
   set_led(2, !led_state[2]); // LED3
   return alt_ticks_per_second() * 1; // 0.5Hz = 2s
}
alt_u32 cb_led5(void* context) {
   set_led(4, !led_state[4]); // LED5
   return alt_ticks_per_second() / 2; // 1Hz = 1s
}
alt_u32 cb_led7(void* context) {
   set_led(6, !led_state[6]); // LED7
   return alt_ticks_per_second() / 4; // 2Hz = 0.5s
}
alt_u32 cb_led9(void* context) {
   set_led(8, !led_state[8]); // LED9
```

```
    return alt_ticks_per_second() / 8; // 4Hz = 0.25s
}

int main() {
    IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0x00);


    alt_alarm_start(&alarm1, alt_ticks_per_second() * 2, cb_led1, NULL);
    alt_alarm_start(&alarm3, alt_ticks_per_second() * 1, cb_led3, NULL);
    alt_alarm_start(&alarm5, alt_ticks_per_second() / 2, cb_led5, NULL);
    alt_alarm_start(&alarm7, alt_ticks_per_second() / 4, cb_led7, NULL);
    alt_alarm_start(&alarm9, alt_ticks_per_second() / 8, cb_led9, NULL);

    while (1) {
            if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x01){
                    is_SW1_SET = 1;
            }
            else {
                    is_SW1_SET = 0;
            }
    }
    return 0;
}
```

Ex2.mp4

Exercise 3:

Code C

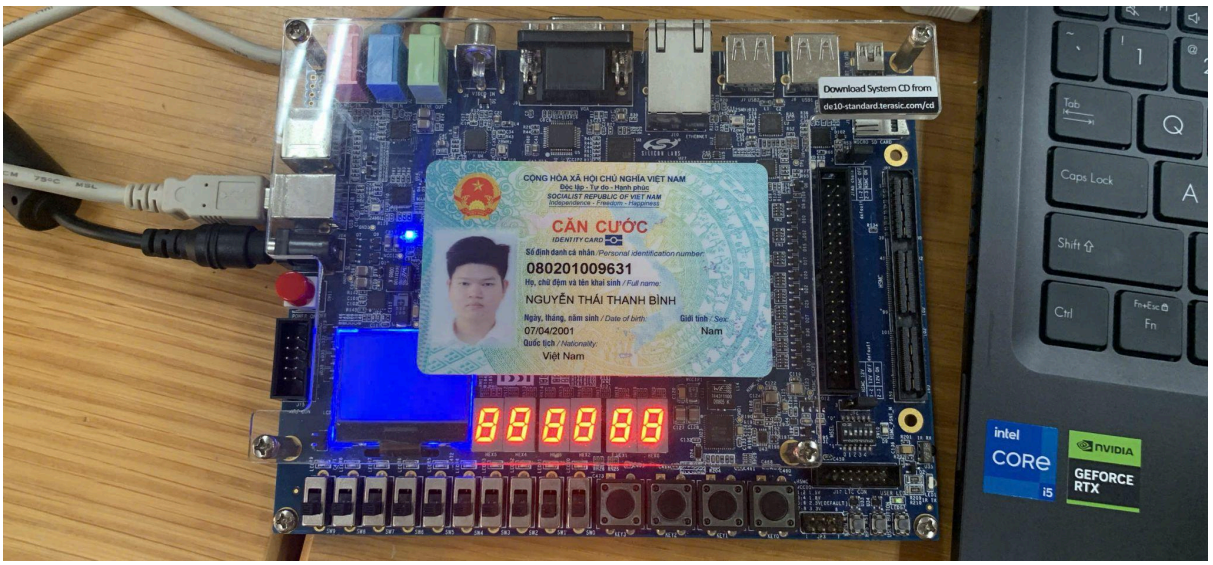
```
#include <stdio.h>
#include <alt_types.h>
#include <sys/alt_alarm.h>
#include "system.h"
#include "altera_avalon_pio_regs.h"

// Define BASE address
#define LED_BASE 0x04081050
#define SW_BASE 0x04081040


static alt_alarm alarm_even, alarm_odd, alarm_sw1_5, alarm_sw6_10;
```

```
static int led_state[10] = {0};
static int even_step = 0;
static int odd_step  = 0;
static int sw1_5_step = 0;
static int sw6_10_step = 0;


void set_led(int index, int state) {
    int mask = 1 << index;
    int current = IORD_ALTERA_AVALON_PIO_DATA(LED_BASE);

    if (state)
        current |= mask;   // bật LED
    else
        current &= ~mask;  // tắt LED

    IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, current);
    led_state[index] = state;
}

alt_u32 cb_even(void* context) {
    if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x01) {
        if (even_step == 5) {
                        even_step = 0;
            int current = IORD_ALTERA_AVALON_PIO_DATA(LED_BASE);
            int mask = (1<<1) | (1<<3) | (1<<5) | (1<<7) | (1<<9);
            current &= ~mask; // Set 0 for even led
            IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, current);
                } else {
            int led_index = (even_step * 2) + 1; // 1,3,5,7,9 → LED2,4,6,8,10
            set_led(led_index, 1);
                        even_step++;
        }
    }
    return alt_ticks_per_second() / 2; // 500ms
}

alt_u32 cb_odd(void* context) {
    if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x01) {
        if (odd_step == 5) {
```

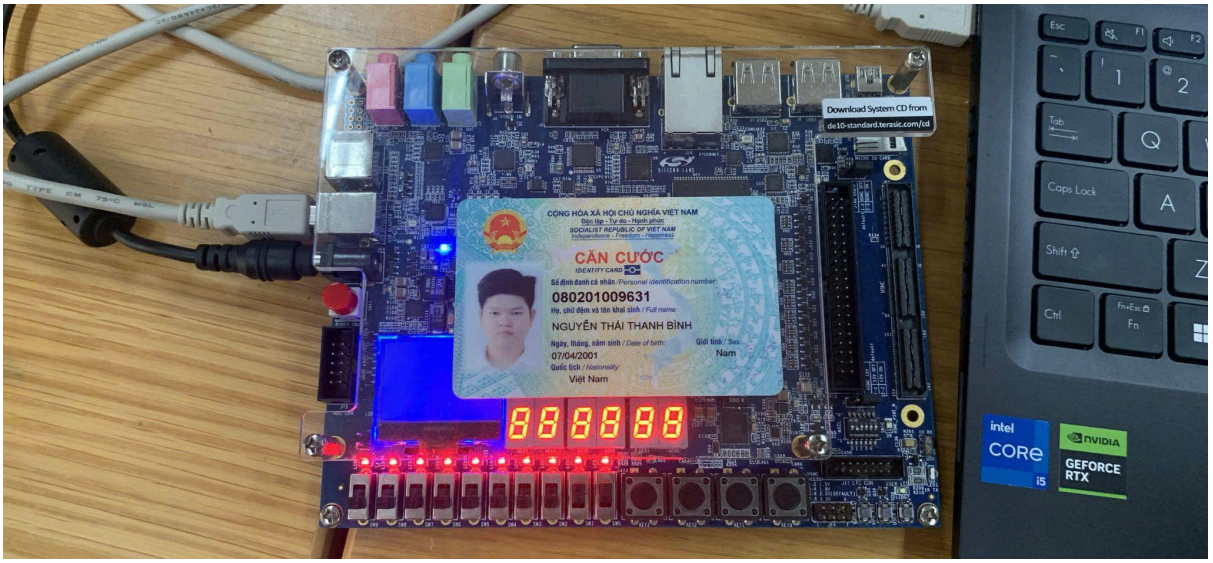
```
                    odd_step = 0;
        int current = IORD_ALTERA_AVALON_PIO_DATA(LED_BASE);
        int mask = (1<<0) | (1<<2) | (1<<4) | (1<<6) | (1<<8);
        current &= ~mask; // Set 0 for odd led
        IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, current);
            } else {
        int led_index = (odd_step * 2); // 0,2,4,6,8 → LED1,3,5,7,9
        set_led(led_index, 1);
                    odd_step++;
    }
  }
  return alt_ticks_per_second(); // 1s
}


alt_u32 cb_sw1_5(void* context) {
  if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x02) {
    if (sw1_5_step == 5) {
                    sw1_5_step = 0;
        int current = IORD_ALTERA_AVALON_PIO_DATA(LED_BASE);
        int mask = (1<<0) | (1<<1) | (1<<2) | (1<<3) | (1<<4);
        current &= ~mask; // Set 0 for LED 1 - 5
        IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, current);
            } else {
        int led_index = sw1_5_step;
        set_led(led_index, 1);
                    sw1_5_step++;
    }
  }
  return alt_ticks_per_second() / 2; // 500ms
}

alt_u32 cb_sw6_10(void* context) {
  if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x02) {
    if (sw6_10_step == 5) {
                    sw6_10_step = 0;
        int current = IORD_ALTERA_AVALON_PIO_DATA(LED_BASE);
        int mask = (1<<5) | (1<<6) | (1<<7) | (1<<8) | (1<<9);
        current &= ~mask; // Set 0 for LED 6 - 10
        IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, current);
```

```
                } else {
            int led_index = sw6_10_step + 5;
            set_led(led_index, 1);
                        sw6_10_step++;
        }
    }
    return alt_ticks_per_second(); // 1s
}

int main() {
    IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0x00);

    // Khởi tạo alarm
    alt_alarm_start(&alarm_even, alt_ticks_per_second() / 2, cb_even, NULL);
    alt_alarm_start(&alarm_odd, alt_ticks_per_second(), cb_odd, NULL);
    alt_alarm_start(&alarm_sw1_5, alt_ticks_per_second() / 2, cb_sw1_5,
NULL);
    alt_alarm_start(&alarm_sw6_10, alt_ticks_per_second(), cb_sw6_10,
NULL);

    while (1) {
        if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x03) {
            IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0x3FF);
            for (int i = 0; i < 10; i++) led_state[i] = 1;
        } else if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x01) {
                        // Decision in call back even/odd
        } else if (IORD_ALTERA_AVALON_PIO_DATA(SW_BASE) == 0x02) {
                        // Decision in call back 1-5, 6-10
        } else {
            IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0x00);
            for (int i = 0; i < 10; i++) led_state[i] = 0;
        }
    }

    return 0;
}
```

Both switches is not set



Both switches set



Switch 1 set:

Ex3_sw1_set.mp4

Switch 2 set:

Ex3_sw2_set.mp4